SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES SIMATS ENGINEERING

LIST OF EXPERIMENTS

SUB CODE & NAME: CSA17- ARTIFICIAL INTELLIGENCE

1. WRITE A PYTHON PROGRAM TO SOLVE THE 8-PUZZLE PROBLEM. (CO1)

```
from collections import deque

def solve(start, goal):
    queue = deque([(start, [])])
    visited = {tuple(start)}

    while queue:
        state, path = queue.popleft()

        if state == goal:
            path.append(state)
            for s in path:
                print(s[:3])
                print(s[3:6])
                print(s[6:])
                print()
            return

        zero = state.index(0)
        row, col = divmod(zero, 3)

        for r, c in [(row-1, col), (row+1, col),
                     (row, col-1), (row, col+1)]:
            if 0 <= r < 3 and 0 <= c < 3:
                pos = r * 3 + c
                new_state = state[:]
                new_state[zero], new_state[pos] = new_state[pos], new_state[zero]

                if tuple(new_state) not in visited:
                    visited.add(tuple(new_state))
                    queue.append((new_state, path + [state]))

    print("No solution exists.")

start = []
goal = []

print("Enter the initial state (use 0 for blank):")
for i in range(3):
    start.extend(map(int, input().split()))
```

```
print("Enter the goal state:")
for i in range(3):
    goal.extend(map(int, input().split()))

print("\nSolution steps:")
solve(start, goal)
```

```
>>> ================= RESTART: C:/Users/deepa/Downloads/AI EXP 1.py ================
    Enter the initial state (use 0 for blank):
    1 2 3
... 4 0 6
... 7 5 8
    Enter the goal state:
    1 2 3
... 4 5 6
... 7 8 0

    Solution steps:
    [1, 2, 3]
    [4, 0, 6]
    [7, 5, 8]

    [1, 2, 3]
    [4, 5, 6]
    [7, 0, 8]

    [1, 2, 3]
    [4, 5, 6]
    [7, 8, 0]

>>>
```

2. WRITE A PYTHON PROGRAM TO SOLVE THE 8-QUEEN PROBLEM. (CO1)

```
def solve(board, row):
    if row == 8:
        for r in board:
            print(" ".join("Q" if x else "." for x in r))
        return True

    for col in range(8):
        if all(
            board[i][col] == 0 and
            (row - i != abs(col - j))
            for i in range(row)
            for j in range(8) if board[i][j] == 1
        ):
            board[row][col] = 1

            if solve(board, row + 1):
                return True

            board[row][col] = 0

    return False
```

```
board = [[0] * 8 for _ in range(8)]

print("Solution:")
solve(board, 0)
```

```
>>>
========== RESTART: C:/Users/deepa/Downloads/AI EXP 1.py ==========
Solution:
Q . . . . . . .
. . . . Q . . .
. . . . . . . Q
. . . . . Q . .
. . Q . . . . .
. . . . . . Q .
. Q . . . . . .
. . . Q . . . .
>>>
```

3. WRITE A PYTHON PROGRAM TO SOLVE THE WATER JUG PROBLEM. (CO2)

```
from collections import deque

def water_jug(capacity1, capacity2, target):
    queue = deque([(0, 0)])
    visited = set()

    while queue:
        x, y = queue.popleft()

        if (x, y) in visited:
            continue

        visited.add((x, y))
        print(x, y)

        if x == target or y == target:
            print("Target reached!")
            return

        next_states = [
            (capacity1, y),
            (x, capacity2),
            (0, y),
            (x, 0),
            (x - min(x, capacity2 - y),
             y + min(x, capacity2 - y)),
            (x + min(y, capacity1 - x),
             y - min(y, capacity1 - x))
        ]

        for state in next_states:
            if state not in visited:
                queue.append(state)

    print("Solution not possible.")
```

```
a = int(input("Enter capacity of Jug 1: "))
b = int(input("Enter capacity of Jug 2: "))
target = int(input("Enter target amount: "))

print("Steps (Jug 1, Jug 2):")
water_jug(a, b, target)
```

```
>>>
========== RESTART: C:/Users/deepa/Downloads/AI EXP 1.py ==========
Enter capacity of Jug 1: 4
Enter capacity of Jug 2: 3
Enter target amount: 2
Steps (Jug 1, Jug 2):
0 0
4 0
0 3
4 3
1 3
3 0
1 0
3 3
0 1
4 2
Target reached!
>>>
```

4. WRITE A PYTHON PROGRAM TO SOLVE THE CRYPT-ARITHMETIC PROBLEM. (CO2)

```
from itertools import permutations

def solve():
    letters = "SENDMORY"

    for p in permutations(range(10), len(letters)):
        d = dict(zip(letters, p))

        if d['S'] == 0 or d['M'] == 0:
            continue

        send = 1000*d['S'] + 100*d['E'] + 10*d['N'] + d['D']
        more = 1000*d['M'] + 100*d['O'] + 10*d['R'] + d['E']
        money = 10000*d['M'] + 1000*d['O'] + 100*d['N'] + 10*d['E'] + d['Y']

        if send + more == money:
            print("SEND =", send)
            print("MORE =", more)
            print("MONEY =", money)
            return

solve()
```

```
>>>
========== RESTART: C:/Users/deepa/Downloads/AI EXP 1.py ==========
SEND = 9567
MORE = 1085
MONEY = 10652
>>>
```

5. WRITE A PYTHON PROGRAM TO SOLVE THE MISSIONARIES AND CANNIBALS PROBLEM. (CO3)

```
from collections import deque

def solve():
start = (3, 3, 1)
goal = (0, 0, 0)
```

```
queue = deque([(start, [])])
visited = {start}

moves = [(1, 0), (2, 0), (0, 1), (0, 2), (1, 1)]

while queue:
    state, path = queue.popleft()
    m, c, boat = state

    if state == goal:
        for step in path + [state]:
            print(step)
        return

    for dm, dc in moves:
        if boat == 1:
            nm, nc = m - dm, c - dc
            nb = 0
        else:
            nm, nc = m + dm, c + dc
            nb = 1

        if not (0 <= nm <= 3 and 0 <= nc <= 3):
            continue

        rm, rc = 3 - nm, 3 - nc

        if (nm > 0 and nm < nc) or (rm > 0 and rm < rc):
            continue

        new_state = (nm, nc, nb)

        if new_state not in visited:
            visited.add(new_state)
            queue.append((new_state, path + [state]))

print("No solution exists.")

print("Steps (Missionaries, Cannibals, Boat):")
solve()
```

```
>>>
========== RESTART: C:/Users/deepa/Downloads/AI EXP 1.py ==
Steps (Missionaries, Cannibals, Boat):
(3, 3, 1)
(3, 1, 0)
(3, 2, 1)
(3, 0, 0)
(3, 1, 1)
(1, 1, 0)
(2, 2, 1)
(0, 2, 0)
(0, 3, 1)
(0, 1, 0)
(1, 1, 1)
(0, 0, 0)
>>>
```

6.WRITE A PYTHON PROGRAM TO SOLVE THE VACUUM CLEANER PROBLEM. (CO3)

```
def vacuum_cleaner():
room = {
    'A': input("Enter status of Room A (dirty/clean): ").lower(),
    'B': input("Enter status of Room B (dirty/clean): ").lower()
}

position = input("Enter vacuum position (A/B): ").upper()

while 'dirty' in room.values():
    print("Vacuum is in Room", position)

    if room[position] == 'dirty':
        print("Cleaning Room", position)
        room[position] = 'clean'

    elif position == 'A':
        position = 'B'
        print("Moving to Room B")

    else:
        position = 'A'
        print("Moving to Room A")

print("All rooms are clean!")
print(room)


vacuum_cleaner()
```

```
>>>
========== RESTART: C:/Users/deepa/Downloads/AI EXP 1.py ==========
Enter status of Room A (dirty/clean): dirty
Enter status of Room B (dirty/clean): dirty
Enter vacuum position (A/B): A
Vacuum is in Room A
Cleaning Room A
Vacuum is in Room A
Moving to Room B
Vacuum is in Room B
Cleaning Room B
All rooms are clean!
{'A': 'clean', 'B': 'clean'}
>>>
```

7.WRITE A PYTHON PROGRAM TO IMPLEMENT BREADTH-FIRST SEARCH (BFS). (CO3)

```
from collections import deque

def bfs(graph, start):
    visited = set()
    queue = deque([start])

    while queue:
        node = queue.popleft()

        if node not in visited:
            print(node, end=" ")
            visited.add(node)

            for neighbor in graph[node]:
                if neighbor not in visited:
                    queue.append(neighbor)

graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': [],
    'F': []
}

start = input("Enter starting node: ").upper()

print("BFS Traversal:")
bfs(graph, start)
```

```
>>> ========== RESTART: C:/Users/deepa/Downloads/AI EXP 1.py ==========
Enter starting node: A
BFS Traversal:
A B C D E F
>>>
```

8.WRITE A PYTHON PROGRAM TO IMPLEMENT DEPTH-FIRST SEARCH (DFS). (CO4)

```
def dfs(graph, node, visited):
    if node not in visited:
        print(node, end=" ")
        visited.add(node)

        for neighbor in graph[node]:
            dfs(graph, neighbor, visited)

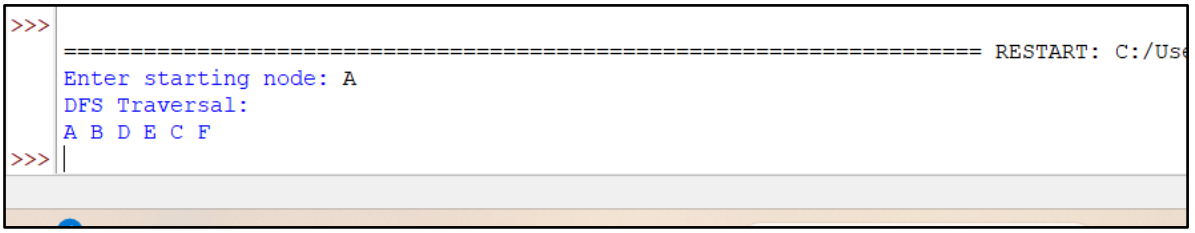

graph = {
```

```
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],


    'D': [],
    'E': [],
    'F': []
}

start = input("Enter starting node: ").upper()

print("DFS Traversal:")
dfs(graph, start, set())
```



9.WRITE A PYTHON PROGRAM TO SOLVE THE TRAVELLING SALESMAN PROBLEM (TSP). (CO4)

```
from itertools import permutations

def tsp(graph, start):
    cities = list(graph.keys())
    cities.remove(start)

    min_cost = float('inf')
    best_path = []

    for path in permutations(cities):
        route = [start] + list(path) + [start]
        cost = 0

        for i in range(len(route) - 1):
            cost += graph[route[i]][route[i + 1]]

        if cost < min_cost:
            min_cost = cost
            best_path = route

    print("Shortest Path:", " -> ".join(best_path))
    print("Minimum Cost:", min_cost)


graph = {
```

```
        'A': {'A': 0, 'B': 10, 'C': 15, 'D': 20},
        'B': {'A': 10, 'B': 0, 'C': 35, 'D': 25},
        'C': {'A': 15, 'B': 35, 'C': 0, 'D': 30},
        'D': {'A': 20, 'B': 25, 'C': 30, 'D': 0}
    }

    start = input("Enter starting city (A/B/C/D): ").upper()

    print("Travelling Salesman Problem:")
    tsp(graph, start)
```

```
>>>
========== RESTART: C:/Users/deepa/Downloads/AI EXP 1.py ==========
Enter starting city (A/B/C/D): A
Travelling Salesman Problem:
Shortest Path: A -> B -> D -> C -> A
Minimum Cost: 80
>>>
```

10.WRITE A PYTHON PROGRAM TO IMPLEMENT THE A* SEARCH ALGORITHM. (CO4)

```
    def astar(graph, heuristic, start, goal):
open_list = [(start, 0)]
cost = {start: 0}
parent = {}

while open_list:
    open_list.sort(key=lambda x: x[1])
    node, _ = open_list.pop(0)

    if node == goal:
        path = []
        while node in parent:
            path.append(node)
            node = parent[node]
        path.append(start)
        path.reverse()

        print("Shortest Path:", " -> ".join(path))
        print("Total Cost:", cost[goal])
        return

    for neighbor, weight in graph[node].items():
        new_cost = cost[node] + weight

        if neighbor not in cost or new_cost < cost[neighbor]:
            cost[neighbor] = new_cost
            parent[neighbor] = node
            f = new_cost + heuristic[neighbor]
            open_list.append((neighbor, f))

print("No path found.")
```

```
graph = {
    'A': {'B': 1, 'C': 4},
    'B': {'A': 1, 'C': 2, 'D': 5},
    'C': {'A': 4, 'B': 2, 'D': 1},
    'D': {'B': 5, 'C': 1}
}

heuristic = {
    'A': 4,
    'B': 3,
    'C': 1,
    'D': 0
}

start = input("Enter starting node: ").upper()
goal = input("Enter goal node: ").upper()

print("A* Search Algorithm:")
astar(graph, heuristic, start, goal)
```

```
>>>
========== RESTART: C:/Users/deepa/Downloads/AI EXP 1.py ==========
Enter starting node: A
Enter goal node: D
A* Search Algorithm:
Shortest Path: A -> B -> C -> D
Total Cost: 4
>>>
```